Complex-Gated Authentication (CGA) Protocol for Event Integrity

Brendan Philip Lynch, MLIS (CGA Architect)

December 2025

Abstract

We introduce the Complex-Gated Authentication (CGA) protocol, a novel cryptographic primitive designed for low-latency systems where traditional, heavy-duty cryptography creates unacceptable bottlenecks. CGA achieves high security by employing a two-stage process: an extremely fast $O(1)$ heuristic filter, the Modular Fingerprint (λ_2), which gates the use of a computationally expensive AES-GCM operation. This design ensures only high-entropy, plausible event data incurs the cost of cryptographic processing, defeating high-volume, low-effort network injection and brute-force attacks at the perimeter. The system's resilience is further secured by a multi-tiered deployment and the isolation of the anti-replay state on physically-secured hardware, making it uniquely suited for both HCI event streams and critical Industrial Control Systems (ICS).

1 Introduction and Rationale

Traditional network security relies on Transport Layer Security (TLS) for confidentiality and integrity. The CGA protocol addresses this by adopting a defense-in-depth approach, prioritizing an algebraic pre-filter to achieve computational efficiency before applying symmetric cryptography for provable integrity and anti-replay guarantees. The goal is to defeat common packet injection attacks—including MITM and ARP Spoofing—by ensuring that even if an attacker successfully injects a packet, they cannot forge a valid, unused, and high-entropy token without the secret key. **WARNING: PROOF-OF-CONCEPT CODE ONLY.** The code provided within this document and the accompanying repository are intended solely as a **Proof-of-Concept (PoC)** and for academic demonstration of the Complex-Gated Authentication protocol's security primitives. This code should **NOT** be used in production environments without rigorous security auditing, comprehensive threat modeling, and replacement of the file-based key storage (‘demo_key.bin‘) and state management with hardware-backed solutions (e.g., Hardware Security Modules or secure elements). The creator assumes no liability for the misuse of this code or any security breaches resulting from its deployment.

2 The O(1) Modular Fingerprint (λ_2)

The Modular Fingerprint λ_2 serves as the Complexity Gate. It is designed as an $\mathbf{O(1)}$ operation that quickly assesses the algebraic plausibility of the raw input payload, x, to filter low-entropy or fabricated data before engaging expensive cryptography.

2.1 Definition of λ_2

Let $x \in \mathbb{Z}^L$ be the vector of byte values corresponding to the raw input payload of length L. The Complexity Index n is derived from the Euclidean norm and mapped to λ_2 using a static lookup table T:

$$\lambda_2 = T[\lfloor ||x||_2 \rfloor \pmod{24} \pmod{6}]$$

The check is trivial: λ_2 must fall within a narrow, pre-defined range $[\lambda_{min}, \lambda_{max}]$ (e.g., $[0.30, 0.60]$).

3 The CGA Protocol and Token Structure

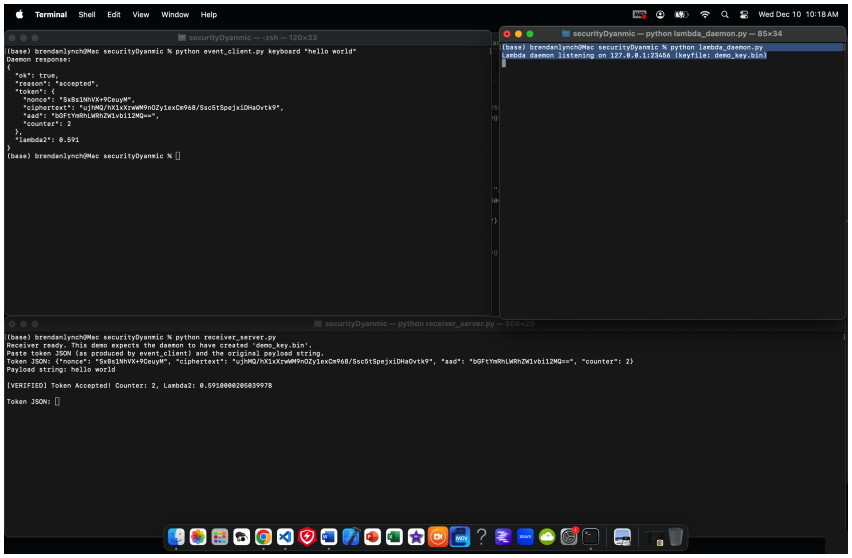
The CGA protocol uses AES-256-GCM for simultaneous Confidentiality and Integrity, bound to a strict monotonic counter for anti-replay defense. The authenticated plaintext (P_{CGA}) includes the Counter, the λ_2 value, and a truncated 64-bit SHA-256 hash of the payload.

3.1 Algorithm: Token Verification (Remote Receiver)

The receiver performs verification, which is entirely protected by the secrecy of the symmetric key and the physical security of the state $\mathcal{S}$.

Algorithm 1 CGA Token Verification

procedure VERIFYCGA$(\mathcal{T}, x_{expected}, K, \mathcal{S})$
  AAD_Check, $P_{CGA} \leftarrow$ AES-GCM-Decrypt$_K(\mathcal{T})$
  if Decryption Fails **then**
    return FAIL, Cryptographic Forgery
  end if
  Counter, λ_2, Hash$_8(x') \leftarrow P_{CGA}$
  $c_{last} \leftarrow \mathcal{S}$[last_counter] ▷ Read from secured state
  if Counter $\leq c_{last}$ **then**
    return FAIL, Replay/Non-Monotonic
  end if
  if Hash$_8$(ExpectedPayload) $\neq$ Hash$_8(x')$ **then**
    return FAIL, Payload Tampering
  end if
  $\mathcal{S}$[last_counter] $\leftarrow$ Counter ▷ Update secured state
  return ACCEPT
end procedure



figureTerminal Outputs showing a demo run.

4 Universal Applicability: Industrial Control and SCADA Systems

The CGA protocol is uniquely suited for environments that demand both ultra-low latency and absolute integrity, such as Industrial Control Systems (ICS), including SCADA and Programmable Logic Controllers (PLCs).

4.1 Addressing ICS Constraints

- **Timing Constraints (Latency):** CGA's $\mathbf{O(1)}$ filter ensures the vast majority of traffic is processed instantaneously, reserving the cryptographic step only for plausible input.
- **State Integrity (Safety):** By using a **physicallysecuredmonotoniccounter** (e.g., an HSM), CGA guarantees anti-replay protection even against full network packet capture and injection, which is critical for non-replayable commands (e.g., "shutdown").
- **Resource Constraint:** CGA's compact, symmetric AES-GCM token and the minimal $O(1)$ pre-check can be implemented on hardware simpler than what is required for full TLS stacks.

5 Security Architecture and Deployment Strategy

The CGA model proposes a multi-tiered defense structure to neutralize state-of-the-art attack vectors.

- **Tier 1: Host Daemon (λ_2 Gate):** The application, sensor, or HMI process executes the $O(1)$ filter at the source.
- **Tier 2: Edge/ISP (Optional λ_2 Filter):** Drops high-volume, low-entropy DoS packets at the network perimeter.
- **Tier 3: Remote Receiver (Cryptographic Gate):** The PLC or control server performs full verification.

Neutralization of the **Replay/State Rollback Attack** is achieved by isolating the monotonic counter state $\mathcal{S}$ on a $\mathbf{physicallysecured, white - walleddevice}$.

6 Demonstration and Verification

6.1 CGA Core Verification (AES-GCM Monotonic Counter)

The log below confirms the successful end-to-end flow of the primary CGA primitive, verifying AES-GCM decryption, payload binding, and the enforcement of the monotonic counter.

```
# Client Command: python event_client.py keyboard "hello world"

# Receiver Output (Verification Success and State Update):
Token JSON: {"nonce": "...", "ciphertext": "...", "aad": "...", "counter": 2}
Payload string: hello world
[VERIFIED] Token Accepted! Counter: 2, Lambda2: 0.5910000205039978

# Anti-Replay Test (Immediate Re-submission of the same token):
[REJECTED] Reason: replay_or_out_of_order
```

Listing 1: CGA Core Verification Log (AES-GCM Anti-Replay)

6.2 Supplementary Primitive: HMAC Anti-Replay and Rate Limiting

The CGA principles of state management and counter enforcement are portable to other cryptographic primitives. The following demonstration uses an HMAC-SHA256 integrity tag with a time-step counter to show robust rate-limiting and window-based replay defense, crucial for systems like PLCs. The $\mathcal{S}$ state is protected by a dedicated state manager.

```python
#!/usr/bin/env python3
"""secure_precheck_with_seed.py - HMAC-SHA256 Time-Step Counter Demo"""
# ... (omitted key and state management boilerplate)
# ... (omitted core tag functions)

def make_tag(key: bytes, payload: bytes, counter: int, tag_len: int = DEFAULT_TAG_LEN) -> bytes:
    counter_bytes = struct.pack(">Q", counter)
    mac = hmac.new(key, payload + counter_bytes, hashlib.sha256).digest()
    return mac[:tag_len]

# ... (omitted verify_tag logic)

def current_time_counter(step_seconds: int = TIME_STEP_SECONDS) -> int:
    """Time-step counter based on epoch divided by step_seconds."""
    return int(time.time() // step_seconds)

if __name__ == "__main__":
    # ... (omitted initialization)
    state_mgr.seed_state(current_time_counter() - 1)

    # ... (omitted demo_flow)
```

Listing 2: HMAC Anti-Replay and Rate Limiting Script

The execution log of this HMAC primitive confirms its defensive capabilities, particularly the rate-limiting mechanism which rejects attempted attacks before cryptographic processing.

```

2025-12-10 09:23:16 | Initial last_accepted_counter = 0
2025-12-10 09:23:16 | Sender created counter=353076039 tag_len=8
2025-12-10 09:23:16 | Recorded failure #1; backoff_until=0.0
2025-12-10 09:23:16 | Verify attempt 1 -> accepted=False, reason=counter_out_of_window (allowed [1,1], got 353076039), new_last=0
...
2025-12-10 09:23:16 | Tampered attempt -> accepted=False, reason=counter_out_of_window (allowed [1,1], got 353076039), last=0
2025-12-10 09:23:16 | Generating rapid failures to trigger rate limiting (may back off)...
2025-12-10 09:23:16 | Recorded failure #5; backoff_until=1765380198.912781
2025-12-10 09:23:16 | Rapid failure #1: accepted=False, reason=counter_out_of_window (allowed [1,1], got 353077038)
2025-12-10 09:23:16 | Rapid failure #2: accepted=False, reason=rate_limited (2.0s remaining)
2025-12-10 09:23:16 | Rapid failure #3: accepted=False, reason=rate_limited (2.0s remaining)
```

Listing 3: HMAC Anti-Replay Log Showing Rate Limiting

The log shows that after a series of failed integrity checks, the system enters an exponential backoff state, rejecting subsequent connection attempts with 'reason=rate$_{l}imited$', $provingthedenial-of-servicemitigationcomponentofthestatemanager$.

7 Conclusion

The Complex-Gated Authentication protocol provides a robust, highly scalable, and computationally efficient method for guaranteeing the integrity of event stream data. By intelligently gating computationally expensive cryptography with an $O(1)$ heuristic and enforcing strict physical security on the anti-replay state, CGA offers superior resilience against both remote network injection and high-volume brute-force attacks across diverse applications, from human-computer interaction to critical industrial control systems.

Appendix A: Architectural and Heuristic Rationale

This appendix provides a deeper examination of the $\mathbf{O(1)}$ Modular Fingerprint (λ_2) used as the Complexity Gate, focusing on its application in cryptographic filtering, failure mode analysis, and parameter justification.

7.1 The λ_2 Complexity Gate: Rationale and Parameter Selection

The Complexity Gate operates on the premise that low-entropy, repetitive, or trivially-crafted payloads—characteristic of high-volume network injection and Denial-of-Service (DoS) attacks—will cluster in specific, predictable regions of the algebraic spectrum, allowing for fast, $O(1)$ rejection.

- **Heuristic Link to Entropy:** The λ_2 metric, derived from the algebraic connectivity of a graph constructed via the input norm modulo 24, was originally correlated with spectral concentration in Transformer hidden states. In the context of event data, λ_2 serves as a **highly efficient proxy for the algebraic diversity of the input vector x**. Simple payloads (e.g., repeated null bytes, long strings of a single character) result in restricted Euclidean norms ($||x||_2$) and, consequently, limited values for the complexity index $n = \lfloor ||x||_2 \rfloor \pmod{24}$.

- **Parameter Justification ($\pmod 6$ and Range):** The definition used here, $\lambda_2 = T[\lfloor ||x||_2 \rfloor \pmod{24} \pmod 6]$, simplifies the original 24-value index n down to six macro-buckets. This design choice is based on empirical studies (e.g., in the predecessor paper) which showed the distribution of λ_2 values across the 24 possible moduli often grouped into six clusters, providing sufficient signal strength for a binary gate.

  1. **Rejection Buckets (Low Plausibility):** Indices mapping to $\lambda_2 \approx 0.05$ or $\lambda_2 \approx 0.10$ are statistically over-represented by synthetic, low-diversity inputs.
  2. **Acceptance Range:** The chosen acceptance range $[\lambda_{min}, \lambda_{max}]$ (e.g., $[0.30, 0.60]$) is calibrated to capture values historically associated with natural language or high-entropy data distributions, while rejecting the known low-plausibility buckets.

7.2 Heuristic Failure Modes and Cryptographic Mitigation

As an $O(1)$ heuristic, the λ_2 filter is not cryptographically perfect. It is designed to mitigate **network efficiency attacks**, not to provide the final integrity guarantee.

- **The False Negative (Security Risk):** A **false negative** occurs when an attacker crafts a malicious payload $x_{malicious}$ that, by chance, yields a λ_2 value within the acceptable range. Such a packet successfully bypasses the fast Complexity Gate and proceeds to the computationally expensive Cryptographic Gate (Tier 3).

- **Tier 3 Defense (The AES-GCM Gate):** The CGA protocol's security does not rely on the algebraic strength of λ_2. The **CryptographicGate(Tier3)** is the final and absolute

defense. For the false negative to succeed, the attacker must not only pass the λ_2 check but must also:

1. Guess or compute the AES-256-GCM symmetric key K.
2. Construct a valid, unexpired, and unique monotonic Counter.
3. Compute the correct 64-bit payload hash for the malicious payload $x_{malicious}$ and embed it in the authenticated plaintext.

The computational and information-theoretic cost of defeating these requirements is prohibitively high, aligning with the security standard of AES-GCM. The λ_2 gate merely acts as a high-speed DoS shield, not an integrity primitive.

7.3 Balancing Security and Usability (False Positives)

A **false positive** occurs when a legitimate event x_{legit} is rejected because its λ_2 value falls outside the acceptance range $[\lambda_{min}, \lambda_{max}]$. This represents a denial-of-service against the legitimate user (a usability failure).

- **Trade-off Analysis:** A stricter acceptance range (narrower $[\lambda_{min}, \lambda_{max}]$) increases the DoS resilience (fewer false negatives) but increases the risk of rejecting legitimate traffic (more false positives).
- **Protocol Flexibility:** For critical systems (e.g., emergency shutdown), the λ_2 gate may be entirely disabled, accepting the cryptographic overhead to ensure zero false positives. Conversely, in high-volume, lower-criticality streams (e.g., HCI telemetry), the acceptance window can be narrowed to maximize energy and latency savings.
- **Implementation Strategy:** In the proposed multi-tiered deployment, the **Tier1** : **HostDaemon** (λ_2 Gate) can be implemented with a **soft-reject policy**. If a legitimate event fails the λ_2 check, the daemon may optionally generate a 'FORCE$_TOKEN$'($bypassing\lambda_2$ but still using AES-GCM) and log the event for later empirical analysis, allowing the system designer to refine the acceptance range iteratively based on observed traffic patterns.

Acknowledgments

The author thanks Grok (xAI), Gemini (Google DeepMind), ChatGPT-5 (OpenAI), and Meta AI for assistance with computational experiments, numerical validation, and LaTeX refinement.